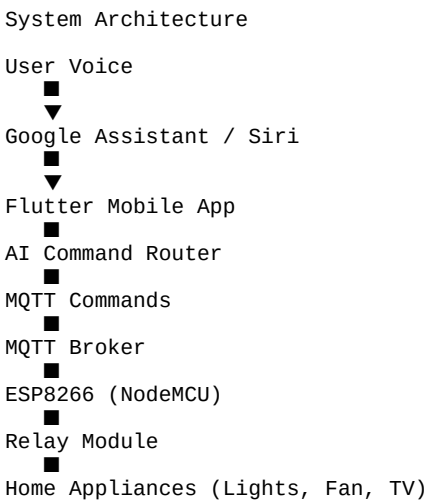


AI Smart Home System Plan (Flutter + ESP8266 + MQTT + Supabase)

This document outlines a professional architecture for building an AI-powered smart home system using Flutter for the mobile application, ESP8266 (NodeMCU) for device control, MQTT for messaging, and Supabase for backend services and device state synchronization.

System Architecture



Flutter Smart Home Dashboard

```
Flutter Smart Home Dashboard

The Flutter app displays a grid dashboard with device cards.

Each card represents a device such as:
- Light 1
- Light 2
- Fan
- TV

Tapping a card toggles the device ON or OFF by sending an MQTT command to the broker.
```

AI Smart Home Brain

```
AI Smart Home Brain

Voice commands are converted into structured actions.

Example:
User command:
"Turn off all lights in the bedroom"

AI Output:
{
  "action": "toggle",
  "devices": ["light1", "light2"],
  "state": "off"
}
```

The router sends MQTT commands to the appropriate device topics.

Voice Control

Voice Control Flow

User speaks a command
↓
Speech recognition converts voice to text
↓
AI router interprets intent
↓
MQTT command is published
↓
ESP8266 receives command
↓
Relay switches appliance

Secure MQTT Architecture

Secure MQTT Architecture

For production deployments, use a secure MQTT broker with:

- TLS encryption
- Device authentication
- Topic-based permissions
- JWT-based user authentication

Recommended brokers:

- HiveMQ
- EMQX

Professional IoT Architecture

Professional IoT Platform Architecture

Mobile App



API Gateway



AI Engine



Device Service



MQTT Broker



IoT Devices (ESP8266 / ESP32)



Relays and Sensors

Additional services:

- Device registry
- Automation engine
- Telemetry service